KORA NEWSROOM · LIVRAISON V1 · FEUILLE DE ROUTE

Aya — ingénieure lead
Semaine du 9 au 13 novembre 2026

À lire avec le **plan commun**, qui fixe le périmètre, les règles, l'ordre de fusion, la recette et le retour arrière.
Ce document détaille tes tâches, avec les commandes exactes et ce qu'il faut vérifier en relecture.

1. Ton rôle cette semaine

- **Intégratrice** : tu es la seule à fusionner dans `main`, dans l'ordre du plan commun (section 5).
- **Relectrice et formatrice** : tu relis les trois PR d'Anne-Laure en binôme et tu l'aides à les mener jusqu'au bout. Les questions de compréhension de la section 4 servent à la faire progresser : l'objectif est qu'elle sache défendre chaque choix de sa PR.
- **Autrice** : PR 0 (lundi), et tes PR #3 et #2 si elles passent les critères.
- **Responsable de la mise en production** jeudi, et de la préparation de la V1.1 (ta pile A3/A4/P0-01).

Quand	Tâche	Section
Lundi 10 h	PR 0, socle CI, fusionnée avant 12 h	2
Lundi 11 h	Rangement des PR de la V1.1 (#4, #6, #7)	3
Lundi après-midi	Pousser le complément de #2 ; préparer #3	5, 6
Mardi 9 h	Relecture de #5 et P2-09, fusion avant 12 h	4
Mardi après-midi	Test comparatif de #2 ; #3 à jour	5, 6
Mercredi 9 h–13 h	Test comparatif et relecture de #1, fusion	4
Mercredi 14 h–18 h	Fusion de #3, décision et fusion de #2, gel	5, 6
Jeudi	Contrôles, publication 11 h, recette	Plan commun 10–11
Vendredi	Étiquette `v1.0.0`, notes de version, V1.1	7, 8

2. Lundi 10 h : PR 0, le socle d'intégration continue

But : que `main` passe au vert pour tout le monde. Tu reprends tes propres corrections, déjà présentes dans #2 et dans la branche C01, sans rien réécrire.

> **Avant la première commande (et à vérifier avec Anne-Laure lundi)**
> Les commandes de ces documents portent des commentaires en fin de ligne. Le zsh de macOS ne les reconnaît pas par défaut et les passe en arguments (`bun run lint # 0 erreur` échouerait). Une fois par poste : `echo 'setopt interactivecomments' >> ~/.zshrc && source ~/.zshrc`

```
git fetch origin
git switch -c chore/socle-ci origin/main

# Réglage ESLint et deux fichiers reformatés : version de ta PR #2
git checkout origin/fix/kora-publication-dates -- \
  eslint.config.js src/server/connectionKeyCrypto.ts src/test/server/budget.test.ts

# Workflow commun (ton commit f94b730 sur la branche C01)
git checkout origin/fix_C01_Securiser_les_sorties_HTTP_du_collecteur -- .github/workflows/ci.yml

bun install --frozen-lockfile
bunx tsc --noEmit -p . && bun run lint && bun --no-env-file run test \
  && bun --no-env-file run build && git diff --check origin/main

git status        # exactement les 4 fichiers ci-dessous, déjà ajoutés par checkout
git add eslint.config.js src/server/connectionKeyCrypto.ts \
  src/test/server/budget.test.ts .github/workflows/ci.yml
git commit -m "Intégration continue : main au vert, fichiers générés exclus du formatage Prettier"
git push -u origin chore/socle-ci
```

Résultat attendu (vérifié le 8 novembre) : ESLint 0 erreur et 6 avertissements `react-refresh` connus, TypeScript OK, 606 tests sur 45 fichiers, build OK.

Description de la PR 0

```
## Problème
`bun run lint` échoue sur main (209 erreurs), presque toutes dans des fichiers régénérés
par la plateforme (src/integrations/supabase/*, appUserConnector.ts). Chaque PR doit donc
reformater ces fichiers pour avoir une CI verte, ce qui crée du bruit et des conflits.

## Changement
- eslint.config.js : règle prettier/prettier désactivée sur les fichiers générés
  (les autres règles restent actives). Repris à l'identique de #2.
- connectionKeyCrypto.ts, budget.test.ts : formatage. Repris de #2.
- .github/workflows/ci.yml : CI commune (TypeScript, ESLint, Vitest, build, espaces).

## Tests
- Lint 0 erreur, 606 tests sur 45 fichiers, build OK.

## Effet sur les autres PR
- #2 et #1 contiennent les mêmes fichiers à l'identique : aucun conflit à prévoir.
- #5 et #1 pourront retirer leur reformatage des fichiers générés.
```

Demande la relecture à Anne-Laure (elle a la grille du plan commun), puis fusionne avec « Create a merge commit ». Préviens-la dès que c'est fait : elle enchaîne sur #5.

> **Avant chaque fusion sur GitHub, toute la semaine**
>
> ☐ CI **quality** verte sur le dernier commit de la PR, et la PR est à jour avec `main` (sinon « Update branch » → « Update with merge commit », jamais « rebase »).
>
> ☐ L'onglet « Files changed » ne montre que les fichiers attendus (listés ci-dessous pour chaque PR), et aucun de `.env`, `drizzle/`, `db/`, `supabase/`, `src/integrations/`.
>
> ☐ Le bouton vert affiche « Create a merge commit » (GitHub retient le dernier choix : vérifier à chaque fois).
>
> ☐ Après la fusion : prévenir l'autre ingénieure, qui met à jour ses branches ouvertes (`git merge origin/main --no-edit`).

> **Après la fusion**
> Le contrôle s'appelle **quality** dans GitHub. Si Amine active la protection de `main` (plan commun, section 15), c'est ce contrôle qu'il faut rendre obligatoire.

3. Lundi 11 h : ranger les PR de la V1.1

☐ Créer l'étiquette **V1.1** (Issues, Labels, New label) et la poser sur #4, #6 et #7.
☐ Passer #4 en brouillon (« Convert to draft », dans la colonne de droite). C'est la seule des trois qui vise directement `main` : elle pourrait être fusionnée par erreur.
☐ Laisser ce commentaire sur #4 :

```
Reportée en V1.1 (semaine du 16 novembre). Cette pile apporte 13 migrations SQL
(0002 à 0014), appliquées à la synchronisation de main : elle demande une fenêtre de
migration avec sauvegarde et vérification préalable. Ne pas fusionner, ne pas changer
la base de #6 et #7 d'ici là.
```

4. Relire les PR d'Anne-Laure

Déroulé de chaque relecture en binôme (30 minutes, partage d'écran) :

1. Anne-Laure présente en 5 minutes : le problème, sa correction, son test qui échouait avant.
2. Vous parcourez ensemble `git diff origin/main` fichier par fichier.
3. Tu poses les questions de compréhension ci-dessous. Si une réponse est floue, vous la travaillez ensemble jusqu'à ce qu'elle soit claire : c'est le cœur de la formation.
4. Tu écris les décisions en commentaire de la PR (une ligne par décision).
5. Si tout est coché : approbation, puis fusion par toi.

4.1 PR #5 — exports CSV (P1-06), mardi 9 h

À vérifier

☐ `git diff origin/main --stat` : seulement `src/server/domain/csv.ts`, `src/test/server/domain/csv.test.ts` et `docs/11-anomalies-connues.md`.
☐ La modification de `package.json` et `bun.lock` (versions de `drizzle-kit` et `drizzle-orm` figées) a disparu : elle était hors sujet.
☐ **Régression corrigée** : avant la PR, une cellule qui commence par une tabulation (`"\tvaleur"`) était préfixée ; la première version de la PR ne le faisait plus, car `trimStart()` retire la tabulation avant le test. La correction attendue et son test sont dans la feuille de route d'Anne-Laure (vérifiés : le test échoue sans la correction et passe avec).
☐ Les nombres restent des nombres (`-42` non préfixé), les chaînes `"-42"` le sont.
☐ P1-06 a quitté le tableau P1 de `docs/11` et figure dans « Corrigées depuis la consolidation ».

Questions de compréhension

Question	Ce qu'on attend
Pourquoi une apostrophe devant la cellule ?	Les tableurs interprètent une cellule qui commence par `=`, `+`, `-`, `@`, tabulation ou retour chariot comme une formule. L'apostrophe force l'affichage en texte. C'est une atténuation, pas une garantie absolue (selon le tableur et le mode d'import).

Pourquoi le bug passait-il avec un tableau ?	Le test était conditionné à `typeof value === "string"` : un tableau d'alias dont le premier élément est `=HYPERLINK(…)` donnait une chaîne finale dangereuse sans être testé.
Pourquoi `−42` n'est-il pas préfixé alors que `"−42"` l'est ?	Un nombre ne peut pas porter de formule ; le préfixer abîmerait les données numériques. Une chaîne peut, elle, venir d'une saisie utilisateur.
Certains tests utilisent `toContain("'")`. Est-ce suffisant ?	Non : ce test passerait même si l'apostrophe était ailleurs dans la ligne. Mieux vaut `toBe(…)` sur la ligne complète. À améliorer si le temps le permet, non bloquant.

4.2 PR P2-09 — fausse source directe, mardi 9 h 30

À vérifier

☐ La PR existe (branche `AL_P2-09_fausse_source_directe`, base `main`).

☐ Le correctif couvre **les deux** endroits fautifs : `src/server/adapters/collector.ts` (flux « propre » de l'entreprise) et `src/server/domain/evidence.ts` (indicateur « source directe »), via une seule fonction `isCompanyHost`.

☐ Tests : le test de collecte existant (`notkalray.eu`) et un test unitaire de `evidence.ts` (`notpasqal.com`).

☐ `docs/11` : P2-09 reste dans le tableau P2 mais ne décrit plus que ce qui reste vrai (filtre par nom par sous-chaîne, sensible aux accents, sans frontière de mot) ; une ligne « P2-09 (partiel) » est ajoutée aux corrigées.

☐ Les quatre autres documents qui décrivaient l'ancien test sont à jour (`docs/03`, `04`, `05`, `06`) : `grep −rn "endsWith(company.domain)\|point de séparation" docs src` ne renvoie que la ligne « P2-09 (partiel) » de `docs/11`.

☐ `git diff origin/main −−stat` : 9 fichiers (`collector.ts`, `evidence.ts`, 2 fichiers de test, `docs/03`, `04`, `05`, `06`, `11`).

☐ **Ordre** : fusionne d'abord #5. Anne-Laure met ensuite P2-09 à jour : un conflit **certain** dans `docs/11` (deux lignes ajoutées au même endroit du tableau « Corrigées »). Vérifie après sa résolution : les deux lignes P1-06 et « P2-09 (partiel) » sont présentes, aucun marqueur, CI verte.

Questions de compréhension

Question	Ce qu'on attend
Pourquoi `host.endsWith(domain)` est-il faux ?	`"notpasqal.com".endsWith("pasqal.com")` est vrai. Il faut l'égalité, ou la fin par `"." + domain` pour un sous-domaine.
Quel effet visible pour un journaliste ?	Un site tiers au nom proche était présenté comme « source directe » de l'entreprise : l'indicateur documentaire montait à tort, et un flux tiers échappait au filtre par nom.
Que reste-t-il de P2-09 après la PR ?	Le filtre par nom des flux généralistes (recherche de sous-chaîne). Prévu en V1.1.

4.3 PR #1 — sorties HTTP du collecteur (P1-08), mercredi

Avant la relecture : test de collecte comparatif (9 h–10 h 30, en binôme)

Protocole et jeu de contrôle dans le plan commun, section 9 (script `test−collecte.sh`). Anne-Laure conduit, tu observes. Attendus : mêmes statuts et mêmes nombres que sur `main`, aucune nouvelle erreur `URL_REFUSED` ni `HOST_MISMATCH`, Alpao en échec `NETWORK` ou `TIMEOUT` rapide, et le port 8443 accepté sur `main` mais refusé sur la branche. Les deux tableaux sont collés dans la PR.

Le conflit de test : à contrôler de près

La mise à jour de #1 avec `main` (après P2-09) produit un conflit imbriqué dans `collecte−reelle.test.ts` : deux morceaux, dont l'un englobe plus de 300 lignes de tests C01. La feuille d'Anne-Laure donne une méthode sûre,

rejouée le 8 novembre (sa version du fichier, puis le test P2-09 recollé à la fin du bloc « flux des entreprises »). Vérifie sur sa branche :

```
grep -c notkalray src/test/server/collecte-reelle.test.ts      # 2
bunx vitest run src/test/server/collecte-reelle.test.ts         # 24 tests verts
```

À vérifier dans le code

☐ Diff limité au sujet : `safeHttp.ts`, `security.ts`, `collector.ts`, leurs tests, la documentation. Plus aucun fichier de `src/integrations/` ; `connectionKeyCrypto.ts`, `budget.test.ts` et `ci.yml` identiques à `main`.

☐ `safeFetchText` : `redirect: "manual"`, revalidation de chaque `Location` par `validateSourceUrl`, 5 redirections au plus, un seul délai de 15 s pour toute la chaîne, corps lu en flux et plafonné à 2 Mio (contrôle de `content-length` puis compteur d'octets).

☐ `validateSourceUrl` : ports 80 et 443 seulement ; nouvelles plages bloquées (192.0.0.0/24, 192.0.2.0/24, 198.18.0.0/15, 198.51.100.0/24, 203.0.113.0/24, 64:ff9b::/96, 2001:db8::/32, fec0::/10). Effet de bord voulu : la création d'une source sur un autre port est refusée en 422 `url_refused`.

☐ `isTransientCollectError` : **changement de comportement à noter**. Avant, tout échec sauf 403 et 404 était réessayé. Maintenant, seuls `TIMEOUT`, `NETWORK`, 408, 425, 429 et 5xx le sont ; un 400, 401 ou 410 ne l'est plus. C'est plus juste ; il faut que ce soit écrit dans la description de la PR.

☐ `HOST_MISMATCH` : refus d'une redirection vers un autre domaine (`www.` mis à part). **Décision : on garde ce comportement strict pour la V1.** Vérifié le 8 novembre : aucune des 30 sources « Site officiel » du pilote ne redirige vers un autre domaine. Il doit être documenté dans `docs/09-exploitation.md`, procédure « Une source est en échec » : corriger l'adresse de la source avec l'adresse finale.

☐ Reporté en V1.1, à ne **pas** faire cette semaine : renommer `timeout_ms` en `timeoutMs` et reprendre `SAFE_FETCH_DEFAULTS.timeoutMs`. Il doit figurer dans « Limites » de la description.

☐ `docs/11` : P1-08 marqué « partiellement corrigé », ce qui reste étant la résolution DNS (rebinding) et le domaine d'entreprise « 127.1 ». `grep -rn "endsWith(company.domain)\|point de séparation\|séparation de domaine" docs src` ne renvoie que la ligne « P2-09 (partiel) ».

☐ `docs/09-exploitation.md` : paragraphe des nouveaux codes d'échec ajouté après la ligne « Symptôme : ».

Questions de compréhension

Question	Ce qu'on attend
Pourquoi `redirect: "manual"` ?	Sinon `fetch` suit les redirections tout seul, sans revalidation : une adresse publique pourrait rediriger vers `127.0.0.1` ou `169.254.169.254` (métadonnées d'un serveur cloud). C'est l'attaque SSRF.
Pourquoi un seul délai pour toute la chaîne ?	Avec un délai par requête, 5 redirections de 15 s chacune bloqueraient la collecte 90 s. C'était l'exercice 4a.
Pourquoi lire le corps en flux ?	`res.text()` charge tout en mémoire : un serveur malveillant ou défaillant peut envoyer des gigaoctets. Le flux permet de couper à 2 Mio.
Pourquoi `URL_REFUSED` n'est-il jamais réessayé, et `TIMEOUT` oui ?	Un refus de sécurité donnera le même résultat à chaque essai et coûte un essai facturé ; un délai dépassé peut être passager.
Qu'est-ce que le DNS rebinding, et pourquoi la PR ne le règle pas ?	Un nom public dont la résolution renvoie une adresse privée. La PR vérifie le nom écrit dans l'adresse, pas l'adresse IP obtenue. Cloudflare Workers bloque en pratique les adresses privées, ce qui limite le risque ; une vraie parade demande une résolution DNS maîtrisée, prévue en V1.1.

Le 5 novembre, un test a été supprimé, puis rétabli le 6 : qu'avait-il de faux ?	Il attendait une erreur sans code ; la règle retenue est « toute erreur porte un code ». Le test a été gardé et son attendu corrigé. Bonne occasion de rappeler : on ne supprime pas un test qui gêne, on comprend pourquoi il échoue.
Quel cas légitime bloque `HOST_MISMATCH`, et que fait l'administratrice ?	Un site qui redirige `societe.com` vers `societe.fr` ou vers `blog.societe.com`. Elle remplace l'adresse de la source par l'adresse finale.

5. PR #2 — dates de publication (décision mercredi 16 h)

Lundi après-midi

☐ Pousser le correctif complémentaire décrit dans la PR : le décodeur utilise `https://news.google.com/rss/articles/{id}` au lieu de `https://news.google.com/articles/{id}`, qui répondait 429 sur le cas Kalray/Openchip. Ajouter le test correspondant si ce n'est pas fait. Mettre à jour la section « non encore poussé » de la description.

☐ `git merge origin/main --no-edit` (après la PR 0) : aucun conflit attendu, les fichiers de la PR 0 sont identiques aux tiens.

☐ **Retirer deux mentions d'outils de génération de code** dans `docs/reference/2026-10-kora-google-news-publication-dates.md` (consigne d'Amine : aucune mention de ce type dans le dépôt) :

- ☐ vers la ligne 183 : « ont été rapportés par Cursor/Grok. » → « ont été ajoutés en dernier. » ;
- ☐ vers la ligne 194 : « `git diff --check` annoncé auparavant par Cursor » → « `git diff --check` sans erreur ».

Contrôle : `grep -n -i -E 'cursor|grok' docs/reference/2026-10-kora-google-news-publication-dates.md` ne doit rien afficher. Les messages de commit déjà poussés ne changent pas (pas de réécriture d'historique) ; ils n'en contiennent pas, vérifié le 8 novembre.

Mardi après-midi : test comparatif

Protocole du plan commun, section 9 (script `test-collecte.sh`), sur `main` puis sur ta branche, à quelques minutes d'intervalle. Le script donne les statuts, le nombre de publications retenues et la durée totale. Pour les publications rejetées et leur motif (ancienne, date non confirmable, dates contradictoires, page éditeur inaccessible), complète avec les journaux du terminal du serveur et l'écran Veille de l'instance locale, sans relancer le script. Repère : le 8 novembre, entre le `main` actuel et la fin de semaine simulée (toutes les PR V1, dont #2), Kalray (Google News) passait de 3 à 2 publications et la durée totale de 8 à 11 secondes.

Mercredi 16 h : décision avec Amine

Critères du plan commun, section 12. Rappel des seuils : rejets des vieilles publications Kalray, aucune source vidée sans motif lisible, au moins 80 % des publications récentes retenues sur `main` encore retenues, durée au plus doublée, CI verte après mise à jour avec `main` (qui contient alors #1).

- **Si oui** : `git merge origin/main --no-edit` (aucun conflit avec #1, vérifié), les cinq contrôles, CI verte, fusion avant 18 h.
- **Si non** : étiquette V1.1, commentaire avec le résultat du test et ce qui manque.

> **Pour la V1.1, dans tous les cas**
>
> `src/server/services/publisher-page.ts` refait son propre `fetch` avec `validateSourceUrl` et `redirect: "manual"`. Une fois #1 fusionnée, il faudra le faire passer par `safeFetchText` (option `maxBytes: 1 Mio`) pour n'avoir qu'une seule porte de sortie HTTP à sécuriser. Ton workflow `kora-quality.yml` fera aussi doublon avec `ci.yml` : à fusionner en un seul.

6. PR #3 — logos (fusion mercredi 14 h si l'identifiant est là)

☐ Obtenir d'Amine l'identifiant client Brandfetch (attendu mardi soir).
☐ Le déclarer dans les variables du projet sur la plateforme sous le nom `VITE_BRANDFETCH_CLIENT_ID`. C'est une variable lue **à la compilation** (`import.meta.env`) : elle n'est prise en compte qu'à la publication suivante. Elle est publique par nature (visible dans le code envoyé au navigateur), ce n'est pas un secret.
☐ `git merge origin/main --no-edit` (aucun conflit, vérifié), puis `bun run lint` **complet** : la description de la PR indique un lint limité aux fichiers modifiés.
☐ **Espace en fin de ligne** dans `docs/12-logos-entreprises-brandfetch.md`, ligne 124 (deux espaces après le titre, saut de ligne Markdown) : `git diff --check origin/main` la signale. Retirer les deux espaces, ou les remplacer par une ligne vide.
☐ La PR ajoute `@playwright/test` en dépendance de développement (tests de navigateur). Sans effet sur la production ; à mentionner dans la description. `bun install --frozen-lockfile` doit passer après la mise à jour.
☐ Vérifier en local, avec puis sans la variable : logos affichés ; sans identifiant, domaine invalide, erreur 404 ou délai de 10 s dépassé, retour aux initiales.
☐ Le workflow `company-logos-ci.yml` ne se déclenche que sur la branche `feat/company-logos` : après fusion il ne servira plus. Le retirer avant fusion ou le garder pour un lancement manuel, au choix ; le test E2E Playwright reste disponible avec `bun run test:e2e`.
☐ Relecture par Anne-Laure avec la grille du plan commun, puis fusion.

7. Jeudi et vendredi : publication et clôture

Jeudi : contrôles, publication à 11 h et recette, selon le plan commun (sections 10 et 11). Tu publies, tu surveilles les journaux pendant la recette, tu animes le bilan de 17 h avec Amine.

Vendredi matin, une fois la livraison validée :

```
git fetch origin && git switch main && git pull --ff-only
git tag -a v1.0.0 -m "KORA Newsroom V1" <sha-du-commit-publié>
git push origin v1.0.0
```

Notes de version (à publier sur GitHub, Releases, depuis l'étiquette)

```
KORA Newsroom V1 — 13 novembre 2026

Sécurité
- Exports CSV : neutralisation des formules étendue aux tableaux et aux valeurs précédées
d'espaces (P1-06).
- Collecteur : chaque redirection est revérifiée, ports 80/443 seulement, réponses plafonnées à 2
Mio,
  délai global de 15 s, plages d'adresses réservées bloquées (P1-08, partiel).

Fiabilité
- « Source directe » : un domaine qui se termine comme celui de l'entreprise n'est plus pris pour
elle (P2-09, partiel).
- [si fusionnée] Collecte : date de publication confirmée sur la page de l'éditeur pour Google
News.

Interface
- [si fusionnée] Logos des entreprises.

Outillage
```

```
- Intégration continue commune, lint vert sur main.

Connu et planifié en V1.1 : traçabilité des appels payants et concurrence (A3/A4), newsletter (P0-01),
P1-03, P1-09, fin de P2-09, protection contre le DNS rebinding.
```

8. Préparer la V1.1 (vendredi après-midi)

1. **Ouvrir les PR manquantes**, en brouillon, pour que la pile soit relisible étage par étage :

   ```
   gh pr create --draft --base fix/kora-a3-tracabilite-appels-payants \
     --head fix/kora-a4-concurrence-reprise --title "KORA A4 — concurrence et reprise"
   gh pr create --draft --base fix/kora-a4-concurrence-reprise \
     --head fix/kora-a4-2-fusion-defusion --title "KORA A4.2 — fusion et défusion"
   ```

2. **Mettre la pile à jour avec `main`** en remontant étage par étage (fusion, pas de rebase) : `a3`, puis `a4-concurrence`, `a4-2`, `a4-3`, `p0-01`. Conflits attendus : `eslint.config.js` (garder la version de `main`, qui couvre aussi `appUserConnector.ts`), `docs/02-architecture.md` et `docs/06-securite-et-flux-tiers.md` (garder les deux apports).
3. **Préparer la fenêtre de migration** :
   - vérifier quelle sauvegarde la base de production a réellement (offre, rétention, restauration à un instant donné) et faire une copie des tables `kora_records`, `kora_settings` et `app_user_connections` juste avant ;
   - faire exécuter `scripts/a43-preflight-readonly.sql` en lecture seule et lire le résultat ;
   - relire le plan de retour arrière (`docs/2026-11-03-a4-3-revue-finale-pr-et-retour-arriere.md`) et le compléter pour `0014` ;
   - choisir un créneau sans utilisateurs, en accord avec Amine. La fusion dans `main` déclenche l'application des migrations : c'est elle, le geste de mise en production.
4. **Consolider les workflows** : `ci.yml`, `kora-quality.yml`, `kora-a3.yml` et `company-logos-ci.yml` en un ou deux fichiers clairs (qualité, plus la base PostgreSQL isolée).
5. **Reste à faire hors pile**, à répartir avec Anne-Laure : lecture des pages éditeur via `safeFetchText` ; P1-03 (appels OpenAI hors budget) ; P1-09 (en-têtes de sécurité, en commençant par HSTS, `X-Content-Type-Options`, `Referrer-Policy` et `frame-ancestors`, la CSP complète en dernier car elle peut casser l'interface) ; fin de P2-09 ; résolution DNS contre le rebinding.

9. Accompagner Anne-Laure

- Elle a une méthode solide : test rouge, correction minimale, test vert, un commit par étape. Valorise-la et garde-la comme norme d'équipe.
- Ses deux points de progrès visibles dans l'historique : le périmètre des PR (reformatage hors sujet, versions de dépendances changées au passage) et la mise à jour d'une branche sans réécrire l'historique. La feuille de route qu'elle a reçue lui donne les commandes exactes ; vérifie avec elle le premier `git merge origin/main` lundi.
- En relecture, demande-lui d'expliquer plutôt que de corriger à sa place. Si elle bloque plus de 30 minutes sur un point, elle doit t'appeler : c'est la règle de la semaine.
- Vendredi, propose-lui de prendre une tâche de la V1.1 en autonomie (par exemple la fin de P2-09, qu'elle connaît déjà).